

Week-9 DL Gen AI

L1 Word Embeddings

- RNNs → good for sequential data.
- Today Gen AI ~ Transformers & Attention mechanism.

- word embeddings → building block of LMs.
- computers operate on numbers, not text. How can we represent the meaning of words in a way a network can process?

way to represent a word as a list of nos. c/d a vector. Each word of the vocab. gets its own unique vector. Think of these vectors as coordinates for each word in a high-dim "meaning space."

- The nos. of vectors aren't random, they capture the semantic meaning & context. Words with similar meaning will have similar vectors. We can compare their distance.
 - some automatic form of clustering will happen.
- The directions btw. vectors can capture relationships. Eg. Male - Female ~ King - Queen.
 - this allows us to manipulation in the semantic space.
- These vectors can't be created by hand. They are learned automatically by algs. that process raw info.

Eg. Word2Vec, GloVe, FastText

- _/_/_
- A word is known by the company it keeps.
 - Foundational Algos $\rightarrow$ generate a single, fixed (word2vec) vector for each word.

It is c/d. a static embedding.

Bank is same for winner & money back.

- Modern Models $\rightarrow$ they generate contextual (BERT, GPT) embeddings. The vector is based on the sentence it is in.

$\rightarrow$ BERT reads all sentence at once. Then develops contextually embeddings.

- Steps -

1. initialize all the vectors randomly.
2. learn jointly $\rightarrow$ Train a NN for some task. We simultaneously adjust the weights.

- Embeddings are the fundamental pillar of modern NLP.

L2 The Attention Mechanism

- RNN Bottlenecks

$\rightarrow$ we had enc-dec. architecture. Final hidden state becomes context vector.

$\rightarrow$ dec. RNN uses the context vector to generate seq.

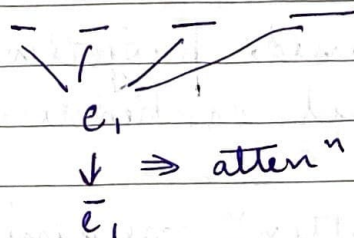
$\rightarrow$ as len inc., we tend to forget.

$\rightarrow$ some early info. tends to dilute.

$\rightarrow$ vanishing / exploding gradients $\Rightarrow$ LSTM, GRU

$\rightarrow$ sequential processing (1-by-1 in RNN) is very slow. At scale, not helpful for us.

- _/_/_
- For early years, LSTMs were SOTA for NLP.
 - The attention mechanism allowed the model to dynamically attend to diff. parts of the input seq. at each step of the output generaⁿ, overcoming the fixed vector limitaⁿ.



- Transformer $\rightarrow$ solely depend on attenⁿ mechanisms.
- Analogy $\rightarrow$ when translating, we don't read the full sentence, memorize it & then write. We focus on diff. parts of the source sentence as they generate each part of the target sequence.
- Attenⁿ allows the dec. to directly look back at the entire source sentence at every step of the output gen. $\rightarrow$ It learns a set of attenⁿ weights for each output word.

$$\downarrow$$
cross-attenⁿ.

- It is a process of mapping a Q (Query) & a set K-V (Key-Value) pairs to an input. This is self-attenⁿ.

$$Q, K, V \rightarrow \left(\begin{array}{l} \text{find the} \\ \text{corresponding} \\ V \text{ value.} \end{array} \right)$$

 (comes in) $\downarrow$ (helps to identify al.)

$Q \rightarrow$ who should I pay attenⁿ to?
 $\rightarrow$ vector representing the current word
 $K \rightarrow$ this is what I am?
 $\rightarrow$ vectors for each word in the sentence
 $V \rightarrow$ this is what I have offer?
they are learned projections of the original word embeddings.

- Main operaⁿ of attenⁿ mechanism is attenⁿ pooling

$q, D = \{(k_1, v_1), \dots, (k_m, v_m)\}$

$$\text{Atten}^n(q, D) = \sum_{i=1}^m \underbrace{\alpha(q, k_i)}_{\text{atten}^n \text{ weights}} v_i$$

$\rightarrow$ we pay attenⁿ to that v_i for which weight α is significant.

- properties of atten weights

1. non-negative wts.
2. convex combinaⁿ (most common)
$$\sum_i \alpha(q, k_i) = 1, \alpha(q, k_i) \geq 0$$
3. hard attenⁿ $\rightarrow$ exactly 1 wt. is 1, others are 0
4. avg. pooling $\rightarrow$ all wts. are equal.
$$\alpha(q, k_i) = \frac{1}{m}$$

- To ensure wts are non-neg & sum to 1 -

1. choose any scoring funcⁿ, $a(q, k)$ that measures the similarity btw. Q & K .

2. apply softmax operaⁿ to these scores to transform them into valid probs.

$$\alpha(q, k_i) = \text{softmax}(a(q, k_i)) = \frac{\exp(a(q, k_i))}{\sum_{j=1}^m \exp(a(q, k_j))}$$

↓ this is ideal as it is differentiable & the gradient never vanishes.

- The similarity btw. k & q can be defined by kernel funcⁿ.

1. Gaussian $\rightarrow \alpha(q, k) = \exp\left(-\frac{1}{2} \|q - k\|^2\right)$

2. Boxcar $\rightarrow \alpha(q, k) = \begin{cases} 1 & \text{if } \|q - k\| \leq 1 \\ 0 & \text{other} \end{cases}$

3. Epanechnikov $\rightarrow \alpha(q, k) = \max(0, 1 - \|q - k\|)$

we can get dot prod. w/o exponentiaⁿ

$$a(q, k_i) = -\frac{1}{2} \|q - k_i\|^2 = q^T k_i - \frac{1}{2} \|k_i\|^2 - \frac{1}{2} \|q\|^2$$

- scoring funcⁿ $\Rightarrow a(q, k_i) = \frac{q^T k_i}{\sqrt{d}}$

↓ Scaled dot product attenⁿ

↓ after this apply normalisation

↓ finally, softmax.

- $q \in \mathbb{R}^{n \times d}$, $K \in \mathbb{R}^{m \times d}$, $V \in \mathbb{R}^{m \times v}$
 n query , m keys of , m values
of len d , len d of len v .

//_

$$\text{Output} = \text{softmax} \left(\frac{QK^T}{\sqrt{d}} \right) V \in \mathbb{R}^{n \times v}$$

2 forms of attenⁿ

1. self-attenⁿ - relates diff. positions of a single seq. to compute a representaⁿ of that same sequence. Allows the model to understand the internal relationships & context within the input.
 → BERT (like contextual)

2. Cross-attenⁿ - relates the posiⁿ of 2 diff. sequences. Connects an enc.-dec. in seq. to seq. tasks.

Self	Cross
= Q; K & V comes from single seq.	- Q comes from 1 seq, K & V comes from 2 nd seq.
- learns int. dependencies & struc. of 1 seq.	- align & integrate info. from 2 diff. seq.

- Issues -

1. variable seq. lens $\Rightarrow$ pad it (shorter ones) but attenⁿ will see
 so mask it.

↳ masked softmax operaⁿ.



limit the attenⁿ summaⁿ to only the valid tokens in the sequence.

L3 Neural Machine Translaⁿ - Overview

- Recap - we checked the impt. of word embedding & attenⁿ mechanism. $\Rightarrow$ Let's apply it on a machine translaⁿ task. (German $\rightarrow$ English)

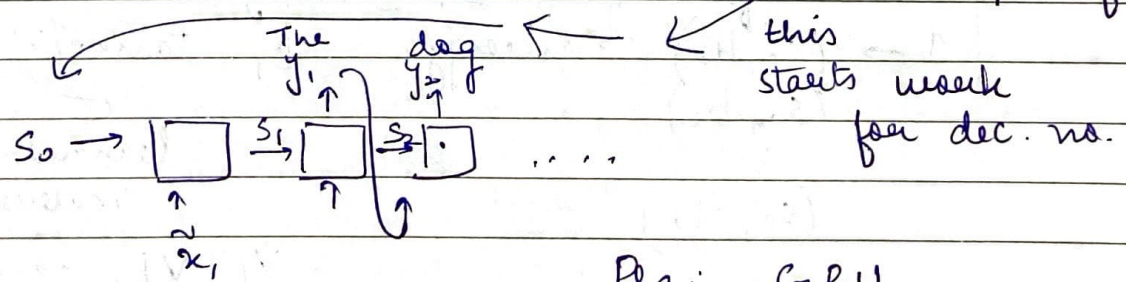
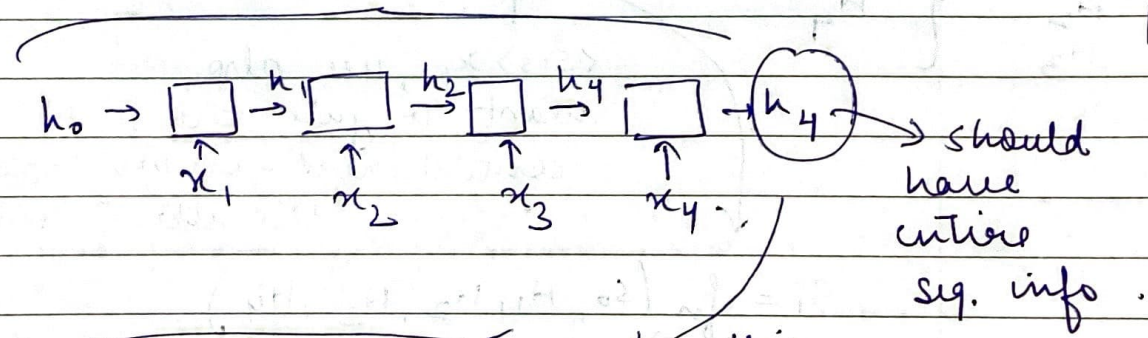
cross-
attenⁿ
on
top of
RNN

many things will be similar to the the IMDB sentiment analysis.

$\hookrightarrow$ here that is GRU (more context window)

Attenⁿ $\rightarrow$ Bahdanau (diff. from scaled dot product attenⁿ)
 $\Downarrow$
good for translaⁿ tasks.

eg. source $\rightarrow$ Der Hund rennt $\langle eos \rangle$
target $\rightarrow$ The dog runs
enc.



s: hidden-decoder
h: hidden-encoder

Plain GRU
with enc. - dec.
architecture

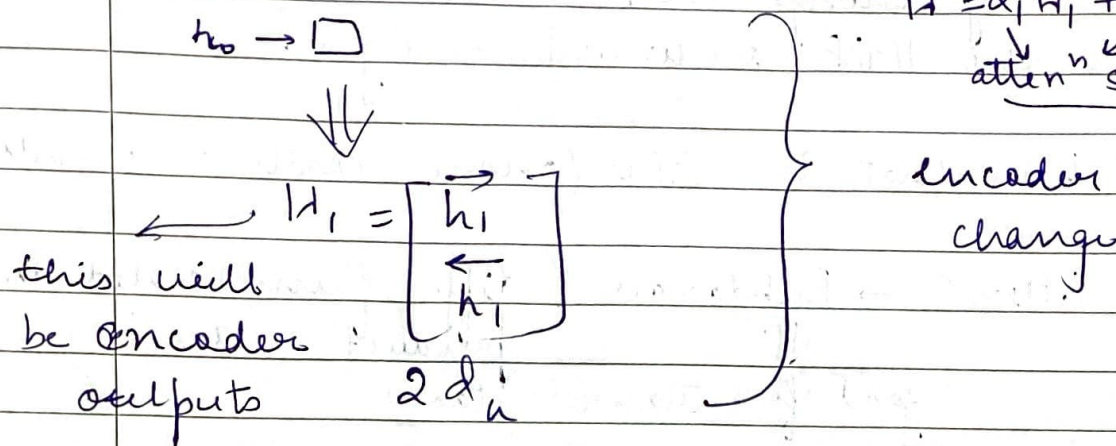
- now there are 2 major changes -

1. we will use bidirectional GRU. because entire source & target is available to us. decoder is still unidirectional.
2. cross-attenⁿ mechanism on top of decoder RNN.

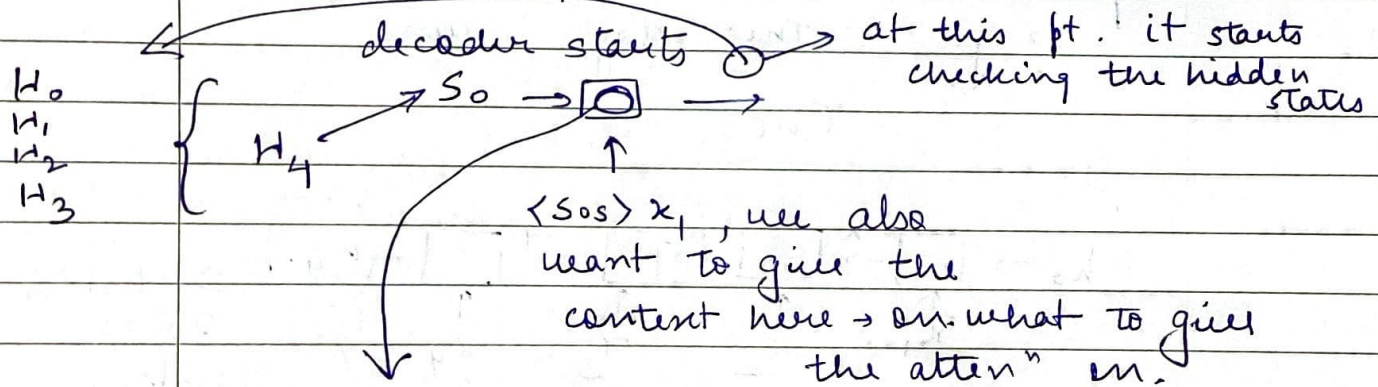
Bahdanau $\Rightarrow$ learn the attenⁿ matrix as well.

$$H = \alpha_1 H_1 + \alpha_2 H_2 + \dots$$

$\downarrow$ $\downarrow$
 attenⁿ scores norm. it.



now, $H_4 \Rightarrow$ encoder hidden state.



$$S_1 = f_n(s_0, H_1, H_2, H_3, H_4)$$

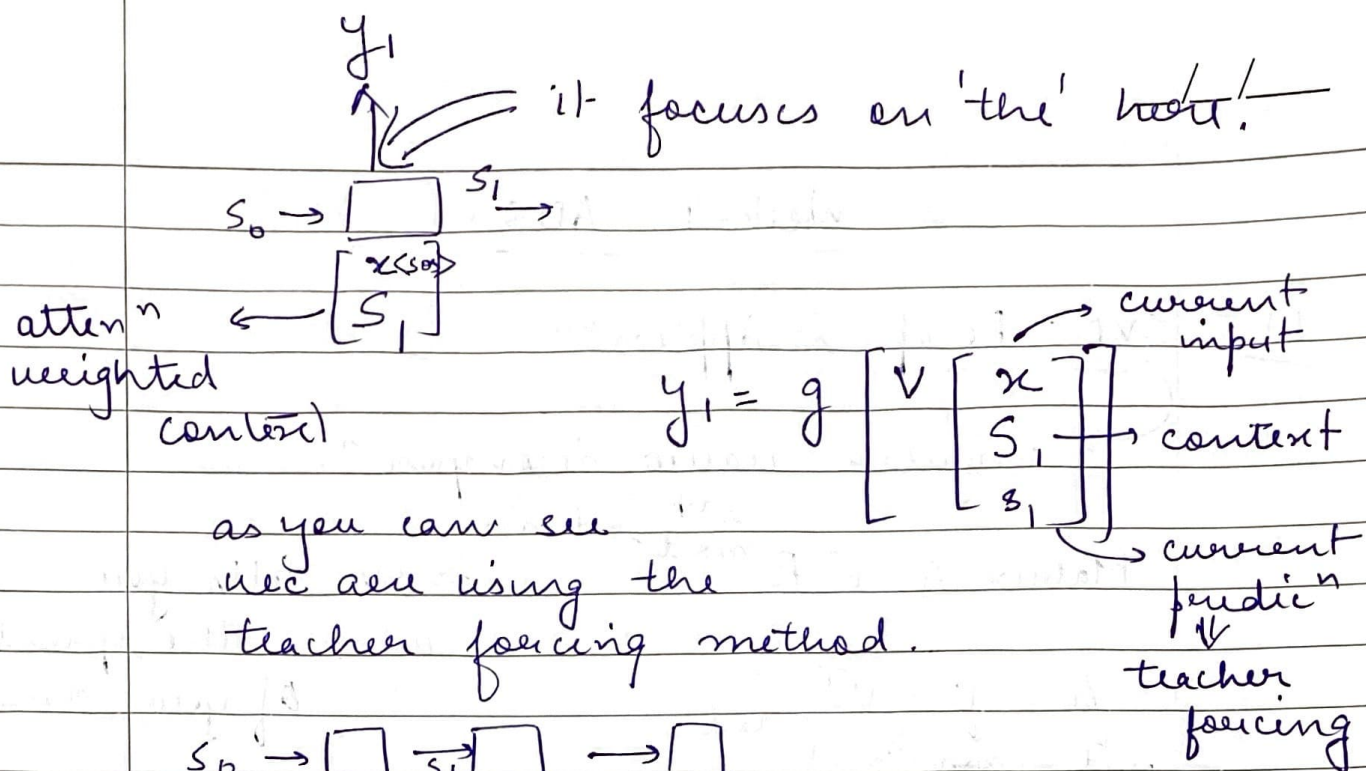
$1 \rightarrow \begin{pmatrix} (s_0, H_1) \\ (s_0, H_2) \\ \vdots \\ (s_0, H_4) \end{pmatrix} \rightarrow \text{energy}$

$$E_1 = \tanh(W_e \begin{bmatrix} s_0 \\ H_1 \end{bmatrix})$$

learned matrix

$$V_1 = V E_1 \Rightarrow \text{scalar}$$

$$2. \rightarrow S_1 = \alpha_1 H_1 + \alpha_2 H_2 + \dots \quad \alpha_1 = \text{softmax}(V_1) \in [0, 1]$$



L4

Code Walkthrough: MT

Enc $\Rightarrow h_1, h_2 \dots h_t = \text{BiGRU}(x_1, \dots, x_t)$

Atten $\Rightarrow a_{ij} = \text{softmax}(\text{score}(s_{j-1}, h_j))$

Context $\Rightarrow c_j = \sum_i a_{ij} \cdot h_j$

Decoder $\Rightarrow s_j = \text{GRU}(y_{j-1}, c_j, s_{j-1})$

- setup the packages (BLEU score)
- download the dataset, check the samples, etc.
- tokenizaⁿ, build vocab
- build the enc. - dec. architecture, then define the attenⁿ mechanism (as discussed earlier)
- then, seq2seq model that combines the enc.-dec.
- instantiate the model and define its configs.
Optim: Adam, Loss: cross entropy, TF: 0.75
- finally, model evaluaⁿ & plotting the inference.